[Final Report Template]

[Block Y1B-2024-25]

Please, use this template to write down your final report for the DataLabTasks. If you have any questions, please, contact your mentor or the content responsible.

Important Notes:

- Please, rename the file to “FinalReport_<your_name>_<studentnumber>.pdf” before submitting it.
- Upload this template to the 'Deliverables' folder in your BUas GitHub repository.
- You are allowed to add as many sub-sections as you need.
- This report will be used for your final presentation in Block Y1B-2024-25 in week 8.
- The final report should only be submitted in “**PDF”** format.

Introduction

This project is about using machine learning to analyze and predict healthcare data. The main goal was to build and compare different models to predict outcomes like how long patients stay in the hospital and the chance of being readmitted. The dataset used had information on patient admissions, medical history, and other important details.

The tasks included exploring and cleaning the data, creating new features, building different models, and evaluating their performance. I worked with several machine learning models like linear regression, decision trees, gradient boosting, XGBoost, logistic regression, random forests, and K-nearest neighbors. I also used techniques like hyperparameter tuning and cross-validation to improve the models.

This report explains the steps I took, the models I tested, and the key findings, showing which approaches worked best for predicting healthcare outcomes using machine learning.

Problem Statement

The goal of this project is to predict important healthcare outcomes, specifically how long patients stay in the hospital and whether they are on diabetes medication. By looking at a dataset with patient details, medical history, and admission information, the aim is to build accurate models that can help healthcare providers make better decisions.

The challenge is to find the best machine learning models that can predict these outcomes correctly. This involves cleaning and processing the data, creating helpful features, choosing the best models, and adjusting them to perform well.

Tasks:

1. Exploratory Data Analysis (EDA) with Python and SQL

1A. Exploratory Data Analysis with SQL

For Task 1A, I performed exploratory data analysis using SQL to understand the structure and distribution of the dataset. Here are the specific steps:

1. Counted the total number of records in the encounter table.
2. Checked the distribution of different admission types.
3. Explored the top discharge dispositions.
4. Checked for missing values in the race column.
5. Checked for missing or unusual values in the weight column.
6. Explored the age distribution of patients.
7. Analyzed how different admission sources contribute to hospital admissions.
8. Investigated which admission types correspond to specific admission sources.
9. Found the average time in hospital for each admission type.
10. Compared discharge dispositions across different admission types.
11. Investigated readmission rates by admission type.

1B. Exploratory Data Analysis with Python

For Task 1B, I performed exploratory data analysis using Python to understand the structure, distribution, and relationships within the dataset. Here are the key steps:

1. Loaded the dataset using pandas and displayed the first and last few rows to understand its structure.
2. Inspected the dataset by checking:
    - Column names
    - Data types of each column
    - Missing values

- Unique values for categorical columns
- Basic statistics for numerical columns
- Shape of the dataset

3. Created visualizations to explore the data:
    - Bar chart showing the number of patients in each age group.
    - Bar chart showing the gender distribution in the dataset.
    - Histogram of time_in_hospital to visualize its distribution.
    - Scatter plot analyzing the relationship between time_in_hospital and num_lab_procedures.
    - Correlation heatmap for key numerical variables such as time_in_hospital, num_lab_procedures, and num_medications.
    - Boxplot analyzing the distribution of num_lab_procedures.
    - Boxplot for time_in_hospital categorized by race.
    - Histogram visualizing the distribution of num_medications.
    - Boxplot for time_in_hospital to identify potential outliers.
4. Summarized findings:
    - Key variables like max_glu_serum and A1Cresult have significant missing values.
    - Some categorical columns have imbalanced distributions, such as gender and age.
    - Numerical columns show distributions, and outliers were identified in columns like time_in_hospital and num_lab_procedures.

2. Data Processing

2.1 Initial Cleaning and Pre-processing

For Task 2.1, I focused on preparing the dataset for ML models by performing initial cleaning steps:

1. Loaded the dataset and displayed its structure using head() and tail().

2. Inspected data types and identified columns with incorrect or inconsistent data types.
3. Checked for duplicate rows in the dataset and confirmed there were none.
4. Dropped columns with high percentages of missing values or those not necessary for the analysis, including:

- max_glu_serum , A1Cresult , weight , payer_code , medical_specialty , encounter_id , patient_nbr , examide ,citoglipton

2.2 Handling Metadata and Missing Values

For Task 2.2, I focused on addressing missing values, mapping categorical data, and ensuring consistency:

1. Identified and replaced occurrences of ? in key columns (race, diag_1, diag_2, diag_3) with their mode values.
2. Mapped categorical columns to numerical values:
    - gender, insulin, change, diabetesMed, readmitted, and medication-related columns were mapped to numeric values.
3. Mapped diagnosis codes (diag_1, diag_2, diag_3) to meaningful categories using a predefined dictionary and encoded them with LabelEncoder.
4. Mapped admission-related columns (admission_type_id, discharge_disposition_id, admission_source_id) to descriptive categories and encoded them using LabelEncoder.
5. Applied one-hot encoding to race and age columns to create binary indicator variables.
6. Converted boolean columns to integers (1 for True, 0 for False).
7. Verified the dataset:
    - Checked for missing values (none remaining).
    - Ensured all columns were appropriately encoded as numerical values.
8. Saved the cleaned dataset as cleaned_data.csv for future use. The dataset now contains 101,766 entries and 52 columns, with no missing values and consistent data types ready for Machine Learning Models.

3. Machine Learning Baselines

3.1 Implementing Regression Baseline

For Task 3.1, I built and tested different regression models to predict the time_in_hospital. Here's what I did:

1. **Data Preparation:**
    - I loaded the cleaned dataset cleaned_data.csv using pandas.
    - I selected important features for prediction, such as:
        - num_medications, num_lab_procedures, number_diagnoses, num_procedures, insulin, change, and discharge_disposition_id.
    - I identified outliers in the dataset by plotting boxplots and analyzing the distribution of key features like num_medications, num_lab_procedures, and time_in_hospital.
    - I scaled numerical columns using StandardScaler to make all features contribute equally.
    - I split the data into training (80%) and testing (20%) sets to evaluate the models.
2. **Models I Built:**
    - **Linear Regression:**
        - I started with a simple linear regression model to set a baseline.
        - I got the following results:
        - MAE: 1.9174
        - MSE: 6.3228
        - RMSE: 2.5145
        - I used cross-validation to check the consistency of the model, and the scores were stable (MAE: 1.915 to 1.933).
        - I visualized the predictions compared to the actual values using a scatter plot.
    - **Ridge Regression:**

- I tested Ridge regression with different values of alpha (1.0, 10.0, 20.0, 50.0).
- The best R^2 score I got was 0.2736, which showed only a slight improvement over the linear regression model.

- **Lasso Regression:**
  - I tried Lasso regression to see if it could select important features.
  - The best R^2 score I got was 0.2717, which was similar to Ridge regression.
- **Decision Tree Regressor:**
  - I implemented a Decision Tree Regressor with parameters max_depth=10 and min_samples_leaf=20.
  - This model performed better than linear models:
  - MAE: 1.833
  - MSE: 5.91
  - RMSE: 2.43
  - This model outperformed linear models. I also visualized the predictions with a scatter plot, showing better alignment with the true values compared to linear models.
- **Gradient Boosting Regressor:**
  - I trained a Gradient Boosting model with n_estimators=500 and learning_rate=0.3.
  - This model performed even better:
  - MAE: 1.7941

    MSE: 5.64
  - RMSE: 2.37
  - Cross-validation showed consistent results (MAE: 1.7567 to 1.8190).
  - I plotted the predictions, which showed a closer fit to the true values.
- **XGBoost Regressor:**
  - I implemented XGBoost with n_estimators=400 and learning_rate=0.05.

        - MAE: 1.7934
        - MSE: 5.63
        - RMSE: 2.37
    - Cross-validation scores were stable (MAE: 1.791 to 1.817).
    - I visualized the predictions, and they were closely aligned with the true values.

3. Summary of Results:

- I created a table to compare the results of all models:

Model	MAE	MSE	RMSE
Linear Regression	1.91	6.32	2.43
Decision Tree	1.83	5.91	2.35
Gradient Boosting	1.7941	5.64	2.37
XGBoost	1.7934	5.63	2.37

4. **Visualizations:**
    - I plotted histograms to check the distributions of selected features.
    - I used scatter plots to compare the true and predicted values for each model.
5. **Key Insights:**
    - Gradient Boosting performed the best, followed by XGBoost.
    - Non-linear models (Decision Tree, Gradient Boosting, XGBoost) outperformed linear models (Linear, Ridge, Lasso Regression).
    - Visualizations showed that the predictions from Gradient Boosting and XGBoost matched the true values better than other models.

3.2 Implementing Classification Baseline

For **Task 3.2**, I built and tested different classification models to predict the target variable diabetesMed. Here's what I did:

1. Data Preparation:

- I loaded the cleaned dataset cleaned_data.csv using pandas.
- I selected important features for prediction, including:
    - time_in_hospital, num_lab_procedures, num_medications, number_outpatient, number_emergency, number_inpatient, and number_diagnoses.
- I used MinMaxScaler to scale the numerical columns, so they all have values between 0 and 1.
- I split the data into training (80%) and testing (20%) sets.

2. Models I Built:

- **Logistic Regression**:
    - I trained a Logistic Regression model with balanced class weights to handle class imbalances.
    - **Evaluation on Test Set**:
        - Accuracy: 0.559
        - Precision: 0.835
        - Recall: 0.536
        - F1 Score: 0.653
    - **Evaluation on Training Set**:
        - Accuracy: 0.5684
        - Precision: 0.8389
        - Recall: 0.5431
        - F1 Score: 0.6593
    - Cross-validation scores were [0.5284, 0.5461, 0.5609, 0.5952, 0.5914].
- **Random Forest**:
    - I used a Random Forest Classifier with max_depth=10, min_samples_leaf=1, min_samples_split=2, and n_estimators=300, with balanced class weights.
    - **Evaluation on Test Set**:
        - Accuracy: 0.6481
        - Precision: 0.8256

        - Recall: 0.6911
        - F1 Score: 0.7524
    - **Evaluation on Training Set**:
        - Accuracy: 0.6720
        - Precision: 0.8430
        - Recall: 0.7048
        - F1 Score: 0.7677
    - Cross-validation scores were [0.6147, 0.6328, 0.6469, 0.6783, 0.6751].
    - This model performed better than Logistic Regression, especially in recall and F1 score.
- **K-Nearest Neighbors (KNN)**:
    - I trained a KNN model with n_neighbors=15 and weights='uniform'.
    - **Evaluation on Test Set**:
        - Accuracy: 0.7683
        - Precision: 0.7858
        - Recall: 0.9630
        - F1 Score: 0.8654
    - **Evaluation on Training Set**:
        - Accuracy: 0.7799
        - Precision: 0.7878
        - Recall: 0.9772
        - F1 Score: 0.8723
    - Cross-validation scores were [0.7606, 0.7644, 0.7647, 0.7665, 0.7667].
    - The KNN model performed the best, especially in recall and F1 score.

3. Summary of Results:

Here's how the models performed:

Model	Accuracy	Precision	Recall	F1 Score

Logistic Regression	0.5624	0.8347	0.5417	0.6570
Random Forest	0.6481	0.8256	0.6911	0.7524
K-Nearest Neighbors	0.7683	0.7858	0.9630	0.8654

5. Key Insights:

- The KNN model was the best in terms of accuracy, precision, recall, and F1 score, followed by Random Forest.
- Non-linear models like KNN and Random Forest performed much better than Logistic Regression.
- KNN and Random Forest produced predictions that were very close to the true values.

3.3 Implementing Clustering Baseline

For **Task 3.3**, I tested different clustering models to group the data points and see how well the clusters worked. Here's what I did:

1. Data Preparation:

- I loaded the cleaned dataset cleaned_data.csv using pandas.
- I selected important features for clustering, like:
    - num_medications, num_lab_procedures, number_outpatient, number_emergency, number_inpatient, gender, admission_type_id, and admission_source_id.
- I removed any duplicate rows in the dataset to keep the data clean.
- I used MinMaxScaler to scale the numerical features so they are between 0 and 1, which is important for clustering.
- I also used PCA (Principal Component Analysis) to reduce the data to 3 main components to help visualize the clusters in 3D.

2. Clustering Methods:

- **K-Means Clustering**:

- I used K-Means clustering with 2 clusters after finding the best number of clusters using the Elbow Method.
- I checked the quality of the clusters using the Silhouette Score, which was 0.6437.
- Visualization: I visualized the clusters in 3D to see how the data points were grouped.

- **Hierarchical Clustering (Agglomerative Clustering)**:
  - I used Agglomerative Clustering with 2 clusters.
  - I trained the model with a sample of 10,000 rows because my laptop lagged a lot so I had to do sample and got a Silhouette Score of 0.6452.
  - Visualization: I visualized the clusters in 3D to show how well the data points were separated into two clusters.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**:
  - I used DBSCAN with eps=0.9 and min_samples=3, which helped the algorithm find noise points.
  - I removed the noise points and got a Silhouette Score of 0.7801 for the remaining clusters.

3. Summary of Results:

Here's how the methods performed:

Clustering Method	Silhouette Score
K-Means	0.6437
Hierarchical	0.6452
DBSCAN	0.7801

4. Key Insights:

- DBSCAN performed the best with the highest Silhouette Score, and the clusters were more clear and well-separated.
- Both K-Means and Hierarchical Clustering gave good results, but they weren't as good as DBSCAN.

3.4 Feature Engineering and Feature Selection

For Task 3.4, I worked on creating new features and choosing the best ones to help the model do better.

1. **Feature Engineering**:
    - I created new features to help improve the models:
        - **Ratios**:
            - admission_discharge_ratio: A ratio between admission_type_id and discharge_disposition_id.
            - admission_source_ratio: A ratio between admission_source_id and admission_type_id.
        - **Interactions**:
            - gender_lab_procedures: A combination of gender and num_lab_procedures.
            - age_medications: A combination of age_[60-70) and num_medications.
        - **Other Features**:
            - total_visits: The sum of number_outpatient, number_emergency, and number_inpatient.
            - medication_per_visit: The number of medications divided by the total visits.
            - change_and_med: A new feature that is 1 when both change and diabetesMed are 1, and 0 otherwise.
    - I also created some statistical features, like medication_per_visit, total_visits, and lab_procedure_procedures_interaction.
2. **Feature Selection**:
    - I used Random Forest to find out which features were the most important for the models.
    - I checked which features were the most useful by visualizing them.
3. **Outlier Handling**:

- I checked for outliers using boxplots and fixed outliers using the IQR method for features like total_visits, medication_lab_interaction, and admission_discharge_ratio.

4. **Model Evaluation After Feature Selection**:
   - **Logistic Regression** had:
     - **Accuracy**: 0.5813
     - **Precision**: 0.8427
     - **Recall**: 0.5609
     - **F1 Score**: 0.6735
   - **Random Forest** had:
     - **Accuracy**: 0.6598
     - **Precision**: 0.8396
     - **Recall**: 0.6900
     - **F1 Score**: 0.7575
   - **KNN** had:
     - **Accuracy**: 0.7685
     - **Precision**: 0.7850
     - **Recall**: 0.9631
     - **F1 Score**: 0.8650
5. **Key Insights**:
     - Logistic Regression showed improvement in accuracy and recall with the new features.
     - Random Forest improved in accuracy, precision, and F1 score with the new features.
     - K-Nearest Neighbors (KNN) showed no change in performance, indicating that the new features did not affect its performance much.

3.5 Hyperparameter Tuning

For Task 3.5, I worked on improving the Logistic Regression model and KMeans clustering through hyperparameter tuning. Here's what I did:

1. **Logistic Regression Hyperparameter Tuning:**
    - I defined a set of hyperparameters for Logistic Regression, such as C (penalty coefficient), penalty, solver, and class_weight.
    - I used GridSearchCV to find the best combination of these hyperparameters by checking the model's accuracy using cross-validation (5-fold).
2. **KMeans Hyperparameter Tuning:**
    - I also set up a hyperparameter grid for KMeans, which included different values for n_clusters (number of clusters), init (initialization method), max_iter (max iterations), and tol (tolerance).
    - However, I faced an issue with the scoring parameter while using GridSearchCV for KMeans. The scoring method I initially chose, silhouette_scorer, was not valid for **GridSearchCV**. This error prevented me from completing the KMeans hyperparameter tuning as planned.
3. **Nested Cross-Validation:**
    - I used nested cross-validation to evaluate the performance of the Logistic Regression model while tuning its hyperparameters. This helped in assessing the model's ability to generalize well on unseen data.
    - The result of the nested cross-validation was an average accuracy of 0.5812, showing that the model was stable and performed decently.
4. **Model Evaluation:**
    - After tuning, I evaluated the Logistic Regression model and got the following results:
        - Train Accuracy: 0.5802
        - Test Accuracy: 0.5813
        - Train F1 Score: 0.6735
        - Test F1 Score: 0.6735
5. **Key Insights:**

- Logistic Regression showed good performance after hyperparameter tuning, with slight improvements in accuracy and F1 score.
- I faced an issue with KMeans hyperparameter tuning due to an invalid scoring parameter. As a result, I couldn't complete the KMeans tuning.
- The nested cross-validation process helped verify that the model was reliable.

3.6 Model Selection

For Task 3.6, I focused on comparing and selecting the best models based on the performance metrics from both regression and classification tasks. The models were evaluated before and after feature engineering, feature selection, and hyperparameter tuning to determine which performed the best in each task.

Regression Model Selection:

Before feature engineering and feature selection, I compared the following models:

Model	MAE	MSE	RMSE
Linear Regression	1.9274	6.3879	2.5274
Decision Tree	1.8338	5.9141	2.4319
Gradient Boosting	1.7942	5.6472	2.3764
XGBoost	1.7935	5.6370	2.3742

After feature engineering and feature selection:

Model	MAE	MSE	RMSE
Linear Regression	1.9274	6.3879	2.5274
Decision Tree	1.8483	6.0036	2.4502
Gradient Boosting	1.7867	5.5779	2.3618
XGBoost	1.7795	5.5498	2.3558

Key Insights for Regression:

- **Gradient Boosting** and **XGBoost** showed the best performance in terms of MAE, MSE, and RMSE. After feature engineering, they performed slightly better, indicating that they can benefit from additional features.
- **XGBoost** had the smallest MAE, MSE, and RMSE values, making it the most accurate model for predicting the time in hospital.
- The **Decision Tree** model performed well initially but had a slight decrease in performance after feature engineering, showing that it may not handle more complex relationships as well as the ensemble models.

Classification Model Selection:

Before feature engineering and feature selection, the results were as follows:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.5594	0.8352	0.5364	0.6532
Random Forest	0.6452	0.8270	0.6846	0.7491
K-Nearest Neighbors	0.7695	0.7850	0.9668	0.8665

After feature engineering, feature selection, and hyperparameter tuning:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.5856	0.8247	0.5864	0.6855
Random Forest	0.7771	0.7814	0.9865	0.8721
K-Nearest Neighbors	0.7707	0.7835	0.9703	0.8670

Key Insights for Classification:

- **Random Forest** outperformed the other models after feature engineering and hyperparameter tuning, showing a significant improvement in Recall and F1 Score. It was particularly good at identifying positive cases, as seen in the high recall value.
- **K-Nearest Neighbors** was consistent and performed well in terms of F1 Score and Accuracy, but Random Forest was slightly better overall.
- **Logistic Regression** showed modest improvements in accuracy, precision, and recall after feature engineering, but it still performed below Random Forest and KNN.

4. Math for Machine Learning

4.1 Implementing Linear Regression with Numpy from Scratch

Cancelled

4.2 Implementing Logistic Regression with Numpy from Scratch

Cancelled

5. Conclusion and Reflection

In this project, I worked on regression and classification tasks using different machine learning models. After performing exploratory data analysis (EDA) and preprocessing, I built and evaluated several models.

Key Findings:

1. **Regression**: XGBoost performed the best in predicting time in hospital, with the lowest error metrics (MAE, MSE, RMSE).
2. **Classification**: Random Forest outperformed the other models, especially in recall and F1 score.
3. **Feature Engineering**: New features like admission_discharge_ratio improved model performance, but KNN remained unchanged after feature engineering.

Reflection:

- Non-linear models like XGBoost (for regression) and Random Forest (for classification) performed better than linear models.
- Hyperparameter tuning improved Logistic Regression, but KMeans tuning was not successful due to scoring issues.
- Future improvements could involve exploring additional models or advanced feature selection techniques.

In conclusion, XGBoost and Random Forest were the best-performing models for the regression and classification tasks, respectively.

6. References

- Scikit-learn Documentation. (n.d.). Retrieved from https://scikit-learn.org/stable/
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- OpenAI. (2025). ChatGPT. Retrieved from https://chat.openai.com/

- DataCamp. (2025). *Python for Data Science and Machine Learning*. Retrieved from https://www.datacamp.com/